

LA RECTIFICATION CONTINUE

Réparation de cohérence JSON comme déploiement opérationnel récursif de la Solution Causale

Logiciel étudié : json-consistency-repair 0.41.0

Solution Causale : <https://zenodo.org/records/21925174>

DOI : <https://doi.org/10.5281/zenodo.21925174>

Note de recherche - 21 août 2026

La version 0.41.0 est la version finale vérifiée PASS041 décrite par la source, les benchmarks et les artefacts de vérification inclus.

Résumé

Ce document présente json-consistency-repair 0.41.0 selon le même modèle de publication que Réparation de cohérence CSV : logiciel ordinaire au point d'usage, documentation de recherche explicite au point d'explication. Le paquet n'est pas seulement un réparateur de JSON malformé. Il découvre des relations stables, reconstruit les valeurs manquantes ou contradictoires lorsque la preuve identifie un terminal unique, applique des patches minimaux et réversibles, remesure l'objet modifié, suit l'autorité et la provenance, tente activement de falsifier les relations apprises, conserve les obligations non closes et vérifie la clôture par rejeu et certification indépendante.

PASS041 ajoute une architecture d'exécution à deux modes. Le mode full effectue l'analyse globale exhaustive, le rejeu et la certification. Le mode fast scanne toujours toutes les familles d'analyseurs, mais limite l'approfondissement coûteux aux cônes locaux d'action uniques et signale explicitement FAST_SCAN_ONLY au lieu de prétendre à une certification complète. La vérification finale consigne 462/462 tests de régression, un stress adversarial PASS041 de 600/600, 73/73 schémas publics Draft 2020-12 valides, des builds wheel/source reproductibles et la parité après installation propre.

Les preuves de benchmark sont volontairement bornées. Elles établissent une récupération exacte sur les corpus déclarés, non une supériorité universelle sur tous les produits de réparation JSON. La comparaison locale directe montre un avantage fort sur la récupération déterministe de JSON malformé et la reconstruction de cohérence sémantique, tandis que les parseurs optimisés restent immensément plus rapides sur du JSON valide ordinaire. Le gate externe restant est une comparaison directe avec des réparateurs JSON tiers spécialisés non installés dans l'environnement de test.

Mots-clés

réparation JSON ; cohérence ; découverte de contraintes ; correction minimale réversible ; provenance ; autorité ; falsification active ; abstention ; rejeu de preuve ; streaming à mémoire bornée ; cohérence distribuée ; calcul causal ; Solution Causale.

Statut du document

Note d'architecture et de preuve orientée publication. Les affirmations numériques sont limitées aux corpus et artefacts de vérification inclus.

Table des matières

1. Objet et position de publication
2. Ce qu'est réellement le logiciel
3. Traduction de la SC en architecture logicielle
4. Le cycle récursif de rectification
5. Pourquoi le JSON comme domaine suivant
6. Architecture de la version 0.41.0
7. Preuves de vérification finale
8. Frontières de preuve sous corruption
9. Modes d'exécution fast et full
10. Frontière syntaxique et ambiguïté
11. Autorité, symétrie et contrôlabilité
12. Falsification, OPEN persistant et cohérence distribuée
13. Information résiduelle et reconstruction aveugle
14. Benchmark comparatif
15. Limites actuelles et affirmations falsifiables
16. Reproductibilité et notes de release
17. Conclusion

1. Objet et position de publication

L'interface reste volontairement conventionnelle. Un utilisateur peut installer un réparateur JSON, inspecter un rapport lisible par machine, utiliser le mode dry-run, traiter des bundles ou des flux, et choisir entre exécution fast et full sans apprendre le vocabulaire de la SC. La note de recherche rend la construction profonde publique plutôt que de la dissimuler.

La séparation est fonctionnelle : outillage JSON ordinaire dans l'interface, architecture causale explicite dans le dossier de recherche.

2. Ce qu'est réellement le logiciel

En surface, json-consistency-repair est un paquet Python sans dépendance runtime tierce pour la réparation JSON récursive et conservatrice. Il couvre le parsing strict, la réparation grammaticale bornée, les objets et tableaux imbriqués, l'arithmétique exacte, les schémas et contraintes DSL, l'identité interdocuments, les bundles, le streaming JSONL/NDJSON et tableaux racine, la planification dry-run, le rejeu inverse et les contrats machine.

Architecturalement, c'est un moteur de rectification sous contraintes. Une mutation doit être soutenue par une relation admissible ou une source d'autorité, franchir les barrières de sécurité pertinentes, améliorer l'état mesuré, rester contrôlable sous permissions et budgets, puis se fermer au rejeu. Lorsque le terminal n'est pas déterminé de façon unique, le moteur préserve l'ambiguïté.

3. Traduction de la SC en architecture logicielle

Notion SC / causale	Réalisation logicielle opérationnelle
Objet	Document JSON, bundle ou flux
Invariant	Schéma, mapping, arithmétique, identité, relation temporelle ou interchamps
Résidu / incohérence	Violation, contradiction, obligation ouverte ou reconstructions conflictuelles
Déplacement admissible	JSON Patch réversible candidat / réparation structurée
Correction minimale	Plus petite mutation ou plan atomique certifié améliorant l'état
Autorité / provenance	Source et périmètre de preuve autorisant une conclusion
Clôture	Point fixe + rejeu + graphe de preuve / certification
Terminal ouvert	Abstention explicite ou témoin manquant typé

4. Le cycle récursif de rectification

- **Observer** : parser le porteur tout en préservant structure, identité et provenance.
- **Découvrir** : inférer les relations stables et compiler schémas, contraintes DSL et rôles des sources.
- **Falsifier** : attaquer activement les relations apprises avant de leur donner une autorité de réparation.
- **Localiser** : ramener violations et obligations ouvertes au plus petit cône causal pertinent.
- **Reconstruire** : générer des terminaux candidats uniquement depuis des preuves admissibles.
- **Contrôler** : vérifier permissions, chemins immuables, budgets et séquences légales de mutations.
- **Matérialiser minimalement** : appliquer le plus petit patch réversible améliorant ou plan atomique.
- **Remesurer** : analyser l'objet physiquement modifié et préserver les relations nouvellement révélées.
- **Vérifier** : contrôler point fixe, rejeu, inverse, graphe de preuve et certification selon le mode.
- **S'abstenir** : laisser l'objet inchangé lorsque la preuve n'identifie pas un terminal unique.

5. Pourquoi le JSON comme domaine suivant

Le JSON est structurellement plus riche que le CSV : hiérarchie, tableaux, identité des objets, scopes imbriqués, références, schémas, sémantique d'ordre, documents multiples et frontières de streaming comptent simultanément. Il teste donc plus sévèrement la capacité des principes de rectification à survivre hors d'une table rectangulaire.

6. Architecture de la version 0.41.0

Couche	Modules représentatifs
Entrée / orchestration	cli.py, engine.py, models.py
Contraintes / calcul exact	constraint_ir.py, constraint_dsl.py, exactmath.py, linear_exact.py
Structure / syntaxe	grammar_repair.py, structural.py, schema_bridge.py, tree.py
Autorité / preuve	authority_scope.py, provenance.py, provenance_chain.py, proof_graph.py, certifier.py
Décision / minimalité	minimal_transfer.py, parallel_routes.py, primary_factorization.py, symmetry.py, controllability.py
Robustesse	relation_falsifier.py, persistent_open.py, horizon.py, information_bounds.py
Échelle / interdocuments	bundle.py, multisource.py, streaming.py, federation.py

7. Preuves de vérification finale

Gate de vérification	Résultat consigné
Suite de régression	462/462 PASS
Stress adversarial PASS041	600/600 ; 0 mutation fausse
Schémas publics Draft 2020-12	73/73 valides
Source extraite propre	PASS040+041 25/25 ; stress 600/600 ; compileall PASS
Wheel installé proprement	pip check PASS ; réparation contrôlée exacte en full + fast
Builds reproductibles	wheel et source reconstruits indépendamment byte-identiques

Ces contrôles établissent la reproductibilité de la release et le contrat déclaré. Ils ne prouvent pas une correction universelle sur des données externes inconnues.

8. Frontières de preuve sous corruption

La campagne de limites distingue l'information manquante de l'information corrompue. Un porteur appris peut cesser d'identifier le régime voulu lorsque plusieurs régimes cohérents restent possibles. C'est une frontière d'identifiabilité, pas automatiquement une limite de capacité du moteur.

Test borné	Corruption	Réparations exactes	Mutations fausses
Implications fonctionnelles autoritatives	39/40 cibles fausses (97,5 %), 3 seeds	117/117	0
Balance arithmétique autoritative	39/40 totaux faux (97,5 %), 3 seeds	117/117	0
Porteur fonctionnel appris	20/40 valeurs manquantes, 3 seeds	60/60	0

9. Modes d'exécution fast et full

Lignes	v0.40 full	0.41 full	0.41 fast	fast vs 0.40
100	1,771 s	0,800 s	0,0275 s	64,4x
500	7,715 s	2,555 s	0,124 s	62,1x
1000	16,622 s	4,496 s	0,250 s	66,5x

10. Frontière syntaxique et ambiguïté

Le contrat syntaxique final PASS041 consigne 14/14 résultats corrects : 13/13 corruptions déterminables réparées et 1/1 ambiguïté de clés dupliquées conflictuelles refusée. L'ensemble réparé comprend virgules manquantes ou finales, deux-points manquant, troncature bornée, guillemet manquant, accolade excédentaire, deux erreurs structurelles bornées, quotes simples, clés non citées et commentaires lorsque la route lexicale bornée converge vers un terminal JSON strict unique.

`{"a":1,"a":2}` n'est pas résolu silencieusement. Sans autorité, deux valeurs dupliquées conflictuelles n'identifient pas laquelle doit survivre.

11. Autorité, symétrie et contrôlabilité

Les passes ultérieures ont déplacé la question de « quelle valeur semble plausible ? » vers « quelle conclusion est réellement autorisée et atteignable ? ». Les rôles des sources et la provenance sont suivis ; une politique externe ne peut pas s'auto-déclarer autoritaire ; des alternatives symétriques bloquent les choix arbitraires par index ; chemins immuables, opérations permises, budgets et précédences déterminent si une réparation identifiable est réellement contrôlable.

Le résultat sépare plus strictement UNKNOWN, identifiable mais inatteignable, atteignable et contrôlable. Une cible mathématiquement connue n'est pas traitée comme applicable sans séquence légale de mutation.

12. Falsification, OPEN persistant et cohérence distribuée

Les relations apprises sont attaquées activement par tests held-out, contrôles négatifs, ablations et recherche de contre-exemples avant promotion. Les relations réfutées sont bloquées avant fédération et optimisation. Les problèmes non résolus persistent comme obligations OPEN typées : source manquante, permission manquante, briseur de symétrie manquant, jointure de réécriture manquante, etc. Elles ne se réveillent que lorsqu'une preuve pertinente arrive.

Le recalcul incrémental est comparé à une exécution complète fraîche. Les contrôles d'horizon et de cohérence distribuée vérifient qu'une conclusion locale ne dérive pas simplement parce que le document, les shards ou la fenêtre de streaming grandissent ; une dérive inexplicée force le recalcul complet ou le conflit plutôt qu'une mutation silencieuse.

13. Information résiduelle et reconstruction aveugle

PASS040 rend mesurable l'insuffisance d'information. Pour un ensemble fini de n candidats restants, le moteur consigne le besoin exact de sélection $\lceil \log_2(n) \rceil$ lorsque la cardinalité est établie ; il n'invente pas une borne de zéro bit lorsque le porteur est inconnu. La reconstruction aveugle retire physiquement la cible avant l'inférence puis vérifie la prédiction seulement après coup contre un engagement haché.

14. Benchmark comparatif

Outil	Récupérations exactes	Acceptations fausses	Refus
Réparation grammaticale JCR	200/200	0	0
JSON5	40/200	0	160
PyYAML safe	40/200	40/200	120
stdlib json	0/200	0	200
orjson	0/200	0	200
ujson	0/200	0	200

Sur le corpus commun de JSON malformé, JCR récupère les 200 cibles déterminables. Le même benchmark ne transforme pas artificiellement des parseurs en réparateurs sémantiques : reconstruction inter-enregistrements, correction autoritative, rejou et abstention sont traités comme différences de capacité, non comme faux scores de 0 %.

La performance reste asymétrique. Sur un document valide de 7 711 octets, orjson mesure environ 271 Mo/s, ujson 220 Mo/s et stdlib json 134 Mo/s, contre 8,48 Mo/s pour le chemin grammatical exact JCR et environ 0,0112 Mo/s pour le moteur complet de cohérence. JCR n'est pas un concurrent de vitesse de parsing brut.

15. Limites actuelles et affirmations falsifiables

- Une comparaison directe avec des réparateurs JSON tiers spécialisés reste un gate externe.
- Une forte corruption sans borne indépendante peut rester réellement sous-déterminée.
- Le mode fast n'est pas une certification complète ; il annonce FAST_SCAN_ONLY.
- PASS est relatif au contrat : aucune violation non résolue sous les relations certifiées, pas vérité externe inaccessible.
- Les limites de ressources/sécurité refusent volontairement l'évidence hors bornes au lieu de la tronquer silencieusement.

16. Reproductibilité et notes de release

Le paquet de release inclut source, wheel, tests, schémas, guides humain/LLM, contrat machine, résultats bruts de benchmark, scripts, rapports historiques de passes et vérification finale. Le wheel et la source sont reproductibles byte pour byte avec le builder de release ; les contrôles sur source extraite et wheel fraîchement installé font partie de la preuve de release.

17. Conclusion

json-consistency-repair 0.41.0 doit être compris comme le déploiement JSON du même programme de rectification rendu d'abord opérationnellement visible dans le CSV, mais sur un porteur structurellement beaucoup plus riche. L'objet central n'est pas un parseur permissif. C'est un cycle qui observe, découvre, falsifie, localise, reconstruit, contrôle, matérialise minimalement, remesure et vérifie.

La séquence de développement a progressivement supprimé les raccourcis qui auraient rendu le résultat facile à surestimer : la provenance peut bloquer l'autorité ; la symétrie peut bloquer un choix arbitraire ; une cible connue peut rester inatteignable ; les relations apprises sont activement attaquées ; les obligations non closes restent explicites ; les résultats incrémentaux sont comparés au recalcul complet ; les changements d'horizon doivent porter un témoin explicatif ; l'information résiduelle est bornée ; la reconstruction aveugle empêche la fuite de la cible ; et le mode fast ne se présente jamais comme une certification complète.

L'énoncé de preuve le plus fort défendable est donc étroit mais substantiel : sur le corpus déterministe inclus de JSON malformé, la couche grammaticale récupère 200/200 cibles sans matérialisation erronée ; avec des bornes indépendantes fortes, les familles fonctionnelle et arithmétique testées réparent exactement 97,5 % des cibles corrompues avec zéro mutation fausse ; le paquet final franchit tous ses gates de vérification de release consignés ; et le chemin fast fournit de fortes accélérations sur le corpus mesuré tout en conservant le même terminal exact. Aucun de ces résultats locaux n'implique une supériorité universelle sur tous les systèmes de réparation JSON.

L'affirmation publique n'est pas « le JSON est résolu ». L'affirmation est qu'un moteur de rectification reproductible existe maintenant, avec décisions, abstentions, périmètre de preuve et gates comparatifs externes explicitement testables.

Coordonnées de release

Logiciel : json-consistency-repair 0.41.0

Python : >= 3.10

Licence : MIT

CLI : json-consistency-repair

Solution Causale : <https://zenodo.org/records/21925174>

DOI : <https://doi.org/10.5281/zenodo.21925174>

Note de construction

La construction de json-consistency-repair a été complétée avec GPT. Solution Causale : <https://zenodo.org/records/21925174> -

DOI : <https://doi.org/10.5281/zenodo.21925174>